

Dynamic visualization

Jonas Schöley



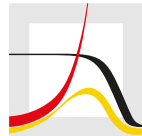
@jschoeley.com



0000-0002-3340-8518



schoeley@demogr.mpg.de



MAX PLANCK INSTITUTE
FOR DEMOGRAPHIC RESEARCH

shiny is:

- 1 A graphical user-interface for your R program
- 2 Your R program as a web-page
- 3 A web-application framework for R



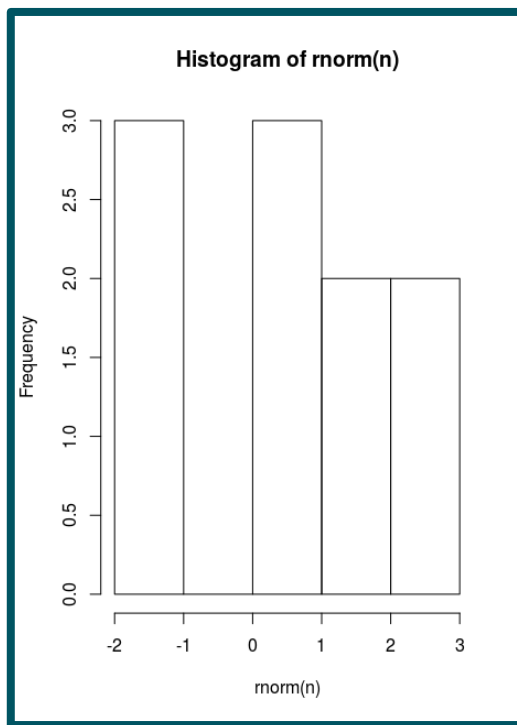
Posit (2025). Shiny: easy web apps for data science. shiny.posit.co/.

Interactivity introducing shiny

shiny is: 1 A graphical user-interface for your R program

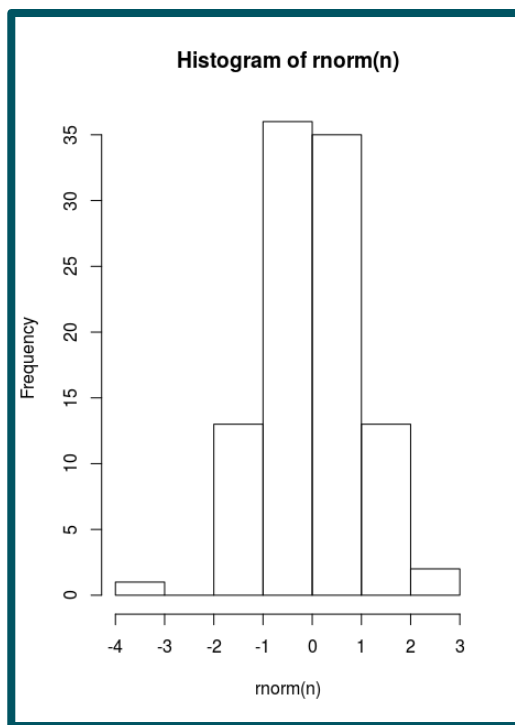
$n = 10$

```
hist(rnorm(n))
```



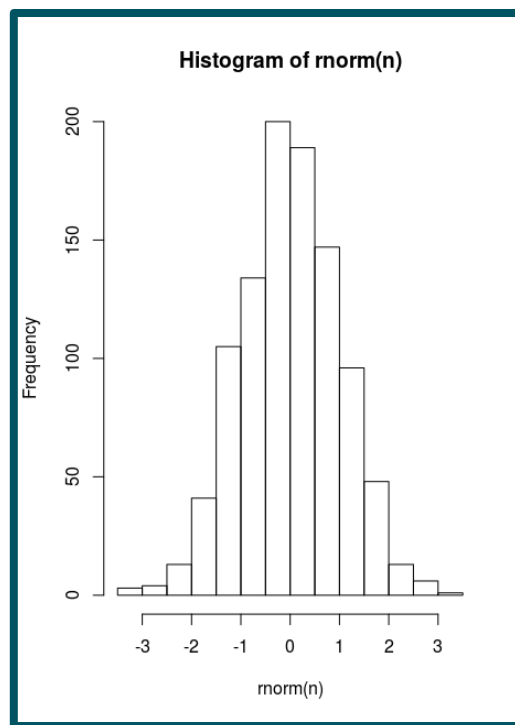
$n = 100$

```
hist(rnorm(n))
```



$n = 1000$

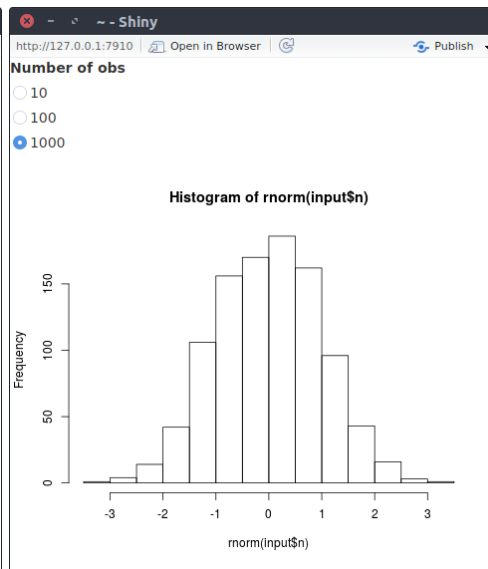
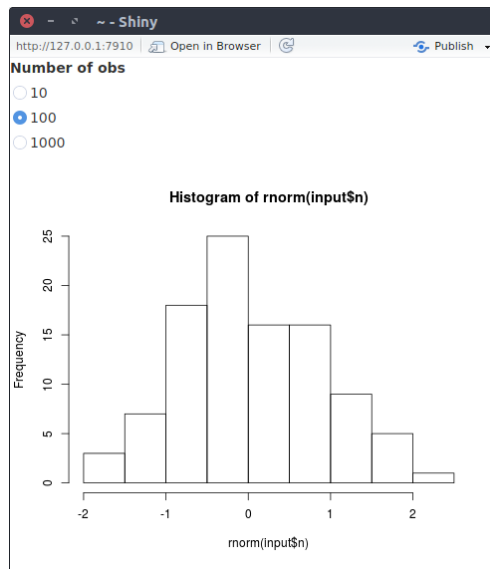
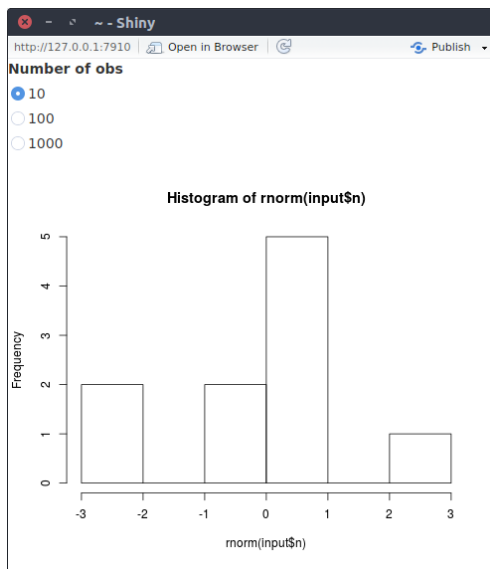
```
hist(rnorm(n))
```



Interactivity introducing shiny

shiny is: 1 A graphical user-interface for your R program

```
library(shiny)
ui <- bootstrapPage(
  radioButtons(inputId = "n", label = "Number of obs",
    choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)
server <- function(input, output) {output$plot <- renderPlot(hist(rnorm(input$n)))}
shinyApp(ui = ui, server = server)
```



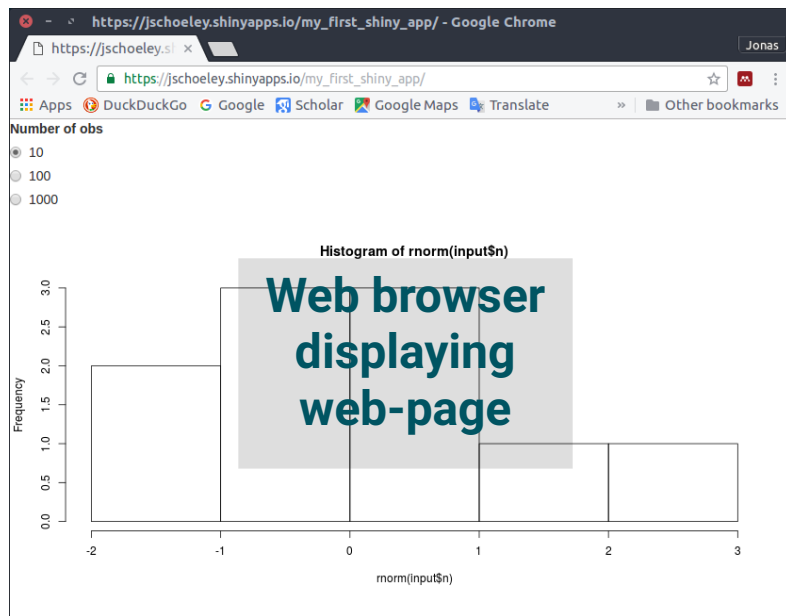
Interactivity introducing shiny

shiny is: **2** Your R program as a web-page



Interactivity introducing shiny

shiny is: **3** A web-application framework for R



- listens for updates from server
- renders web-page

sends user input
to server

sends UI updates
to user

Web server
running
R+Shiny

- listens for user input
- performs R calculations
- builds web-page

Building a shiny app means:

- 1 Building a user interface (UI)
- 2 Connecting R code with the UI

Interactivity introducing shiny

library(shiny)

```
ui <- bootstrapPage(  
  radioButtons(inputId = "n",  
               label = "Number of obs",  
               choices = list(10, 100, 1000)),  
  plotOutput(outputId = "plot")  
)
```

```
server <- function(input, output) {  
  output$plot <- renderPlot(  
    hist(rnorm(input$n))  
  )  
}
```

```
shinyApp(ui = ui, server = server)
```


1 Building a user interface (UI)

```
library(shiny)
```

```
ui <- bootstrapPage(  
  radioButtons(inputId = "n",  
               label = "Number of obs",  
               choices = list(10, 100, 1000)),  
  plotOutput(outputId = "plot")  
)
```

```
server <- function(input, output) {  
  output$plot <- renderPlot(  
    hist(rnorm(input$n))  
  )  
}
```

```
shinyApp(ui = ui, server = server)
```

Interactivity introducing shiny

```
library(shiny)

ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

Create an empty web-page

learn about more sophisticated layouts at
shiny.posit.co/r/articles/build/layout-guide/

Interactivity introducing shiny

```
library(shiny)

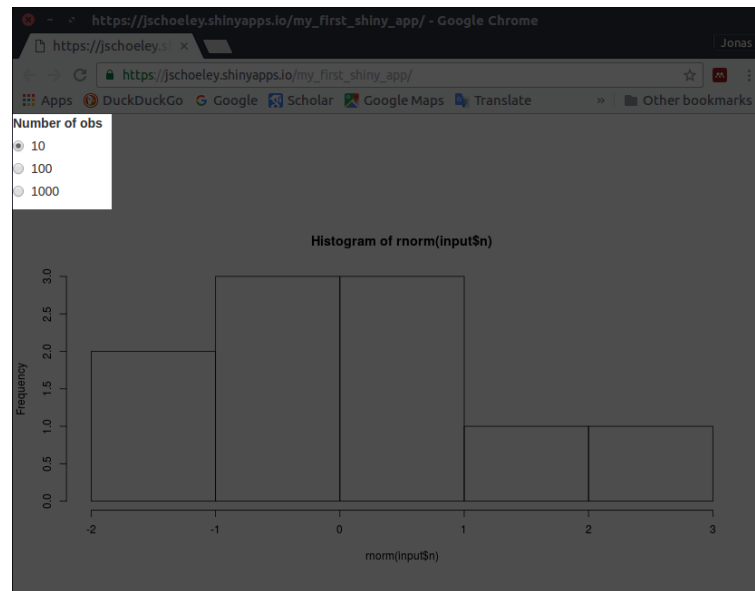
ui <- bootstrapPage(
  radioButtons(inputId = "n",
    label = "Number of obs",
    choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

Define input UI elements

Learn more about different input elements
shiny.posit.co/r/components/



Interactivity introducing shiny

```
library(shiny)
```

```
ui <- bootstrapPage(  
  radioButtons(inputId = "n",  
                label = "Number of obs",  
                choices = list(10, 100, 1000)),  
  plotOutput(outputId = "plot")  
)
```

```
server <- function(input, output) {  
  output$plot <- renderPlot(  
    hist(rnorm(input$n))  
  )  
}
```

```
shinyApp(ui = ui, server = server)
```

Define input UI elements

Learn more about different input elements
shiny.posit.co/r/components/

Inputs

collect values from the user

Access the current value of an input object with `input$<inputId>`. Input values are **reactive**.

Action `actionButton`(inputId, label, icon, ...)

Link `actionLink`(inputId, label, icon, ...)

☒ Choice 1
☒ Choice 2
☐ Choice 3
☒ Check me

checkboxGroupInput(inputId, label, choices, selected, inline)

checkboxInput(inputId, label, value)

dateInput(inputId, label, value, min, max, format, startview, weekstart, language)

dateRangeInput(inputId, label, start, end, min, max, format, startview, weekstart, language, separator)

Choose File

fileInput(inputId, label, multiple, accept)

1

numericInput(inputId, label, value, min, max, step)

passwordInput(inputId, label, value)

☒ Choice A

☐ Choice B

☐ Choice C

Choice 1

Choice 2

radioButtons(inputId, label, choices, selected, inline)

selectInput(inputId, label, choices, selected, multiple, selectize, width, size) (also `selectizeInput()`)

0 5 10

0 2 4 6 8 10

sliderInput(inputId, label, min, max, value, step, round, format, locale, ticks, animate, width, sep, pre, post)

Apply Changes

submitButton(text, icon)
(Prevents reactions across entire app)

Enter text

textInput(inputId, label, value)

Interactivity introducing shiny

```
library(shiny)

ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

Define output UI elements

Learn more about different output elements
shiny.posit.co/r/components/



Interactivity introducing shiny

```
library(shiny)
```

```
ui <- bootstrapPage(  
  radioButtons(inputId = "n",  
    label = "Number of obs",  
    choices = list(10, 100, 1000)),
```

```
  plotOutput(outputId = "plot")
```

```
)
```

```
server <- function(input, output) {  
  output$plot <- renderPlot(  
    hist(rnorm(input$n))  
  )  
}
```

```
shinyApp(ui = ui, server = server)
```

Define output UI elements

Learn more about different output elements
shiny.posit.co/r/components/

Outputs

Outputs create a spot on the webpage to display results from the server, such as text, tables, plots, and more.

The outputs shown here are a small sample of Shiny outputs available in R. For more, see [htmlwidgets](https://htmlwidgets.github.io/).

DataTable

This	That
And	The
Other	Thing

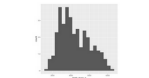
Image



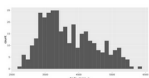
Map (leaflet)



Plot (ggplot2)



Plot (plotly)



Table

species	island	bill_length_mm	bill_depth_mm
Adelie	Torgueman	39.13	18.79
Adelie	Torgueman	39.33	17.45
Adelie	Torgueman	40.33	18.55
Adelie	Torgueman	141	141
Adelie	Torgueman	36.75	19.30

Table (gt)

Penguins in the Palmer Archipelago				
sex	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
Male	37.13	17.79	187.2	5251.0
Female	39.13	18.79	191.2	4901.0
Unknown	40.33	18.55	193.2	4301.0

Table (reactable)

species	island	bill_length_mm	bill_depth_mm
Adelie	Torgueman	39.1	18.7
Adelie	Torgueman	39.3	17.4
Adelie	Torgueman	40.3	18.5
Adelie	Torgueman	36.7	19.3

Text

UI

☒ Show slider



Value Box

Total Sales in Q2
\$2.45M



Verbatim Text

Interactivity introducing shiny

```
library(shiny)

ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

Don't forget the comma

the UI specification is one large nested function call with different UI elements as parameters

Interactivity introducing shiny

```
library(shiny)
```

```
ui <- bootstrapPage(  
  radioButtons(inputId = "n",  
               label = "Number of obs",  
               choices = list(10, 100, 1000)),  
  plotOutput(outputId = "plot")  
)
```

```
server <- function(input, output) {  
  output$plot <- renderPlot(  
    hist(rnorm(input$n))  
  )  
}
```

2 Connecting R code with the UI

```
shinyApp(ui = ui, server = server)
```


Interactivity introducing shiny

```
library(shiny)

ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)
```

```
server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}
```

```
shinyApp(ui = ui, server = server)
```

R communicates with the UI using 2 lists
the **input** list contains the values of all input
elements of the UI, the **output** list contains
the objects which R sends to the UI

2 Connecting R code with the UI

Interactivity introducing shiny

```
library(shiny)

ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

R communicates with the UI using 2 lists
the **input** list contains the values of all input
elements of the UI, the **output** list contains
the objects which R sends to the UI

Interactivity introducing shiny

```
library(shiny)

ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

R communicates with the UI using 2 lists
the **input** list contains the values of all input
elements of the UI, the **output** list contains
the objects which R sends to the UI

Interactivity introducing shiny

```
library(shiny)

ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

The “actual” R code

R calculates new output once new input arrives

Interactivity introducing shiny

library(shiny)

```
ui <- bootstrapPage(  
  radioButtons(inputId = "n",  
               label = "Number of obs",  
               choices = list(10, 100, 1000)),  
  plotOutput(outputId = "plot")  
)
```

```
server <- function(input, output) {  
  output$plot <- renderPlot(  
    hist(rnorm(input$n))  
  )  
}
```

```
shinyApp(ui = ui, server = server)
```

Static R code

R code which is only run once per shiny session goes outside the ui or server functions.

Interactivity introducing shiny

```
library(shiny)

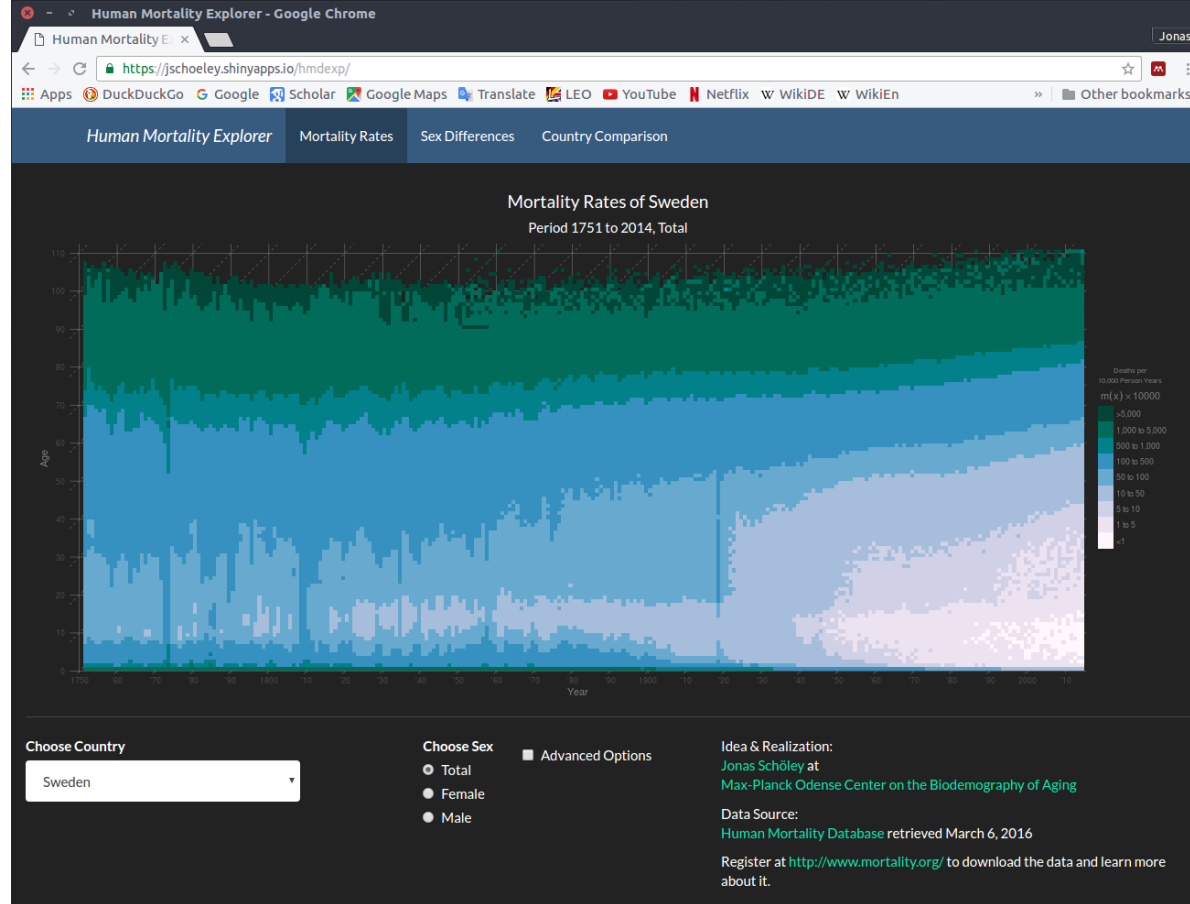
ui <- bootstrapPage(
  radioButtons(inputId = "n",
               label = "Number of obs",
               choices = list(10, 100, 1000)),
  plotOutput(outputId = "plot")
)

server <- function(input, output) {
  output$plot <- renderPlot(
    hist(rnorm(input$n))
  )
}

shinyApp(ui = ui, server = server)
```

Interactivity introducing shiny

Example hmdexplorer.

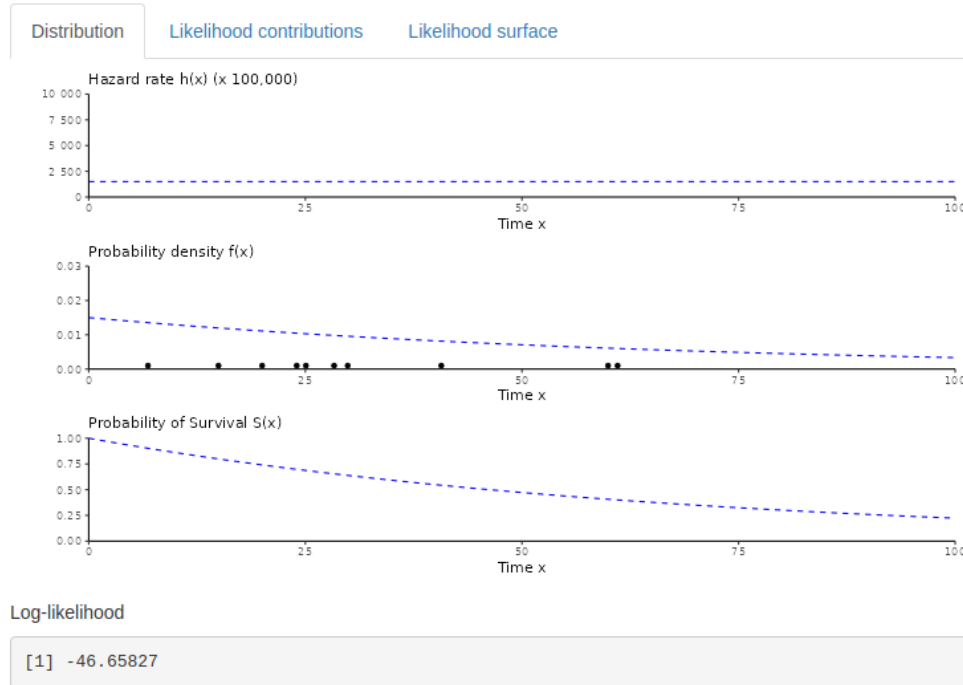
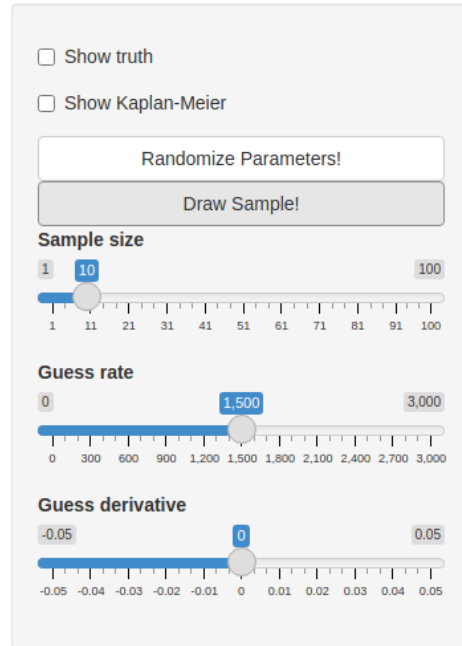


Schöley (2016). The human mortality database explorer. jschoeley.shinyapps.io/hmdexp/.

Interactivity introducing shiny

Example `likelihoodguessr`.

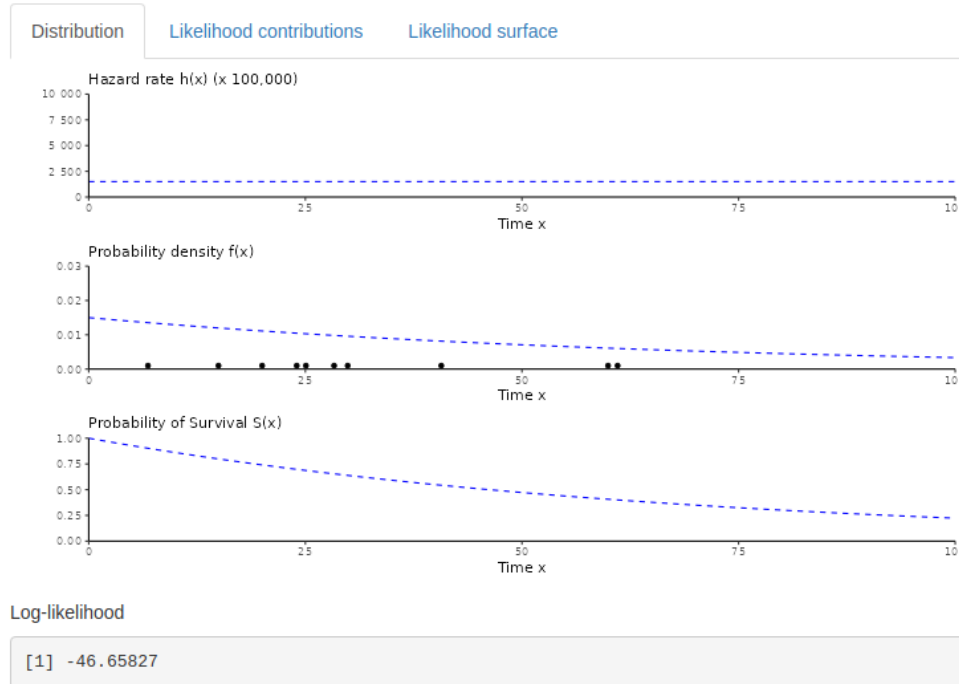
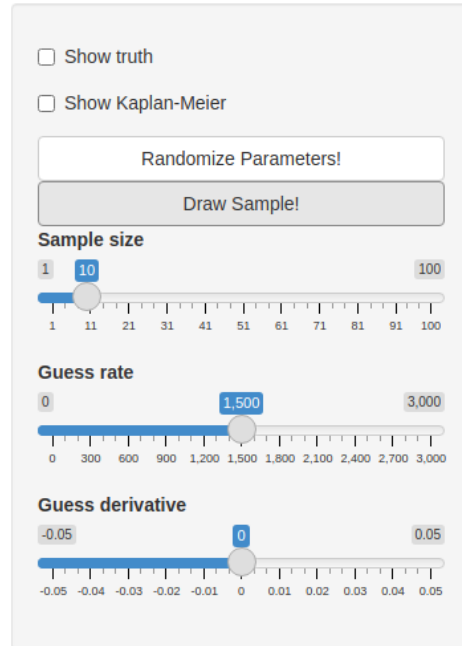
Likelihood maximization



Interactivity introducing shiny

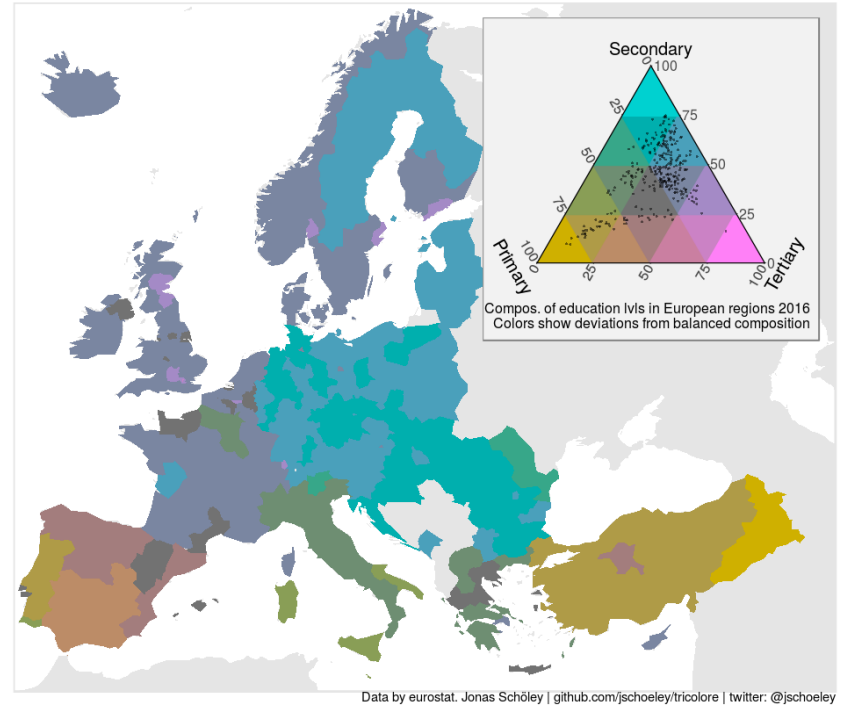
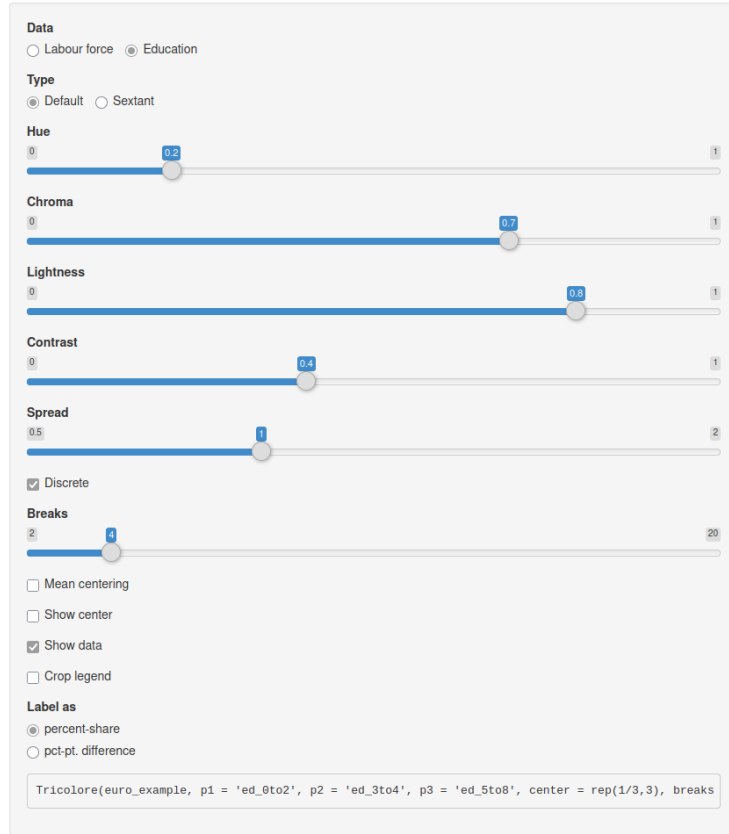
Example `likelihoodguessr`.

Likelihood maximization



Interactivity introducing shiny

Tricolore: A flexible color scale for ternary compositions



Data by eurostat. Jonas Schöley | github.com/jschoeley/tricolore | twitter: @jschoeley

Schöley (2018). tricolore. jschoeley.shinyapps.io/tricolore/.

Reproducible analysis

github.com/jschoeley

Jonas Schöley
jschoeley.com



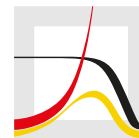
@jschoeley.com



0000-0002-3340-8518



schoeley@demogr.mpg.de



MAX PLANCK INSTITUTE
FOR DEMOGRAPHIC RESEARCH